Topic 20 — Wi-Fi Password Cracking Attack

WPA2 Handshake Capture & Offline Dictionary Attack

A from-scratch WPA2-PSK attack — our own EAPOL parsing and PMK/PTK/MIC cryptography.

No `aircrack-ng`, `hashcat`, `pyrit` or `hcxtools`. `scapy` is used only to sniff and craft raw 802.11 frames — never for the cracking mathematics.

GROUP MEMBERS

Name	Roll
Member One	2105xxx
Member Two	2105xxx
Member Three	2105xxx

Department of Computer Science and Engineering, BUET

Design Report — submitted for evaluation

1 Overview of the Project Idea

Most home and small-office Wi-Fi networks use WPA2-PSK security: every device that joins types in the same shared password (the "Wi-Fi password"). That password is never sent over the air. Instead, when a device connects, it and the access point (AP) run a short exchange called the **4-way handshake** to prove to each other that they both know the password, without either one ever transmitting it in the clear. This handshake is carried inside a Wi-Fi management protocol called **EAPOL** (Extensible Authentication Protocol over LAN).

This project attacks that scheme end to end. It does *not* try to guess the password by talking to the AP directly — that would be slow and would get noticed. Instead, it **captures one handshake off the air**, and then, completely **offline**, tries candidate passwords from a wordlist until one of them reproduces the exact cryptographic proof seen in the captured handshake. This is possible because the handshake contains enough information for anyone holding a candidate password to check, by themselves, whether that password is the right one — the AP does not need to be asked.

1.1 Two phases

The project is split into an **online phase** (needs a wireless card in radio range of the target) and an **offline phase** (needs only the saved capture file, and can run anywhere, later, on any machine):

- **Online** — `capture.py` passively listens for a client completing the 4-way handshake with the target AP and saves it to a file. `deauth.py` is an optional accelerant: it forges Wi-Fi deauthentication frames to knock a connected client offline so it reconnects immediately, instead of waiting for that to happen naturally.
- **Offline** — `handshake.py` parses the saved capture file into candidate handshakes, and `crack.py` runs the actual dictionary attack: for each candidate password, it derives the same keys the AP and client would have derived, and checks whether they match what was captured.

1.2 What is, and is not, reused

The cracking mathematics — pcap parsing, 802.11/EAPOL frame decoding, PMK/PTK derivation, and MIC verification, in `handshake.py` and `crack.py` — is a from-scratch implementation with **zero third-party cracking dependencies**: no `aircrack-ng`, `hashcat`, `pyrit`, or `hcxtools` is invoked anywhere. `capture.py` and `deauth.py` use the `scapy` library for one job only — sniffing and crafting raw 802.11 radio frames — in place of tools like `airodump-ng`/`aireplay-ng`. This distinction matters for the rest of the report: sections 3–5 describe code that this project wrote and controls, not the behaviour of an external tool.

1.3 Scope

The target is **WPA2-PSK** (personal / pre-shared-key) networks specifically, using CCMP (AES) or TKIP pairwise ciphers — the common case for home routers. Enterprise Wi-Fi (802.1X with a RADIUS server, a different key hierarchy) and WPA3-SAE are out of scope; the defenses in section 7 return to why.

2 Network Topology, Components and Timing

Three parties are involved: a legitimate **Client (STA)** that already has the Wi-Fi password and is on the network; the target **Access Point (AP)**, identified by a fixed hardware address called its BSSID; and the **Attacker**, who only needs a wireless network card capable of *monitor mode* within radio range of both — no prior access to the network is required. Monitor mode lets a

wireless card read every frame it can physically hear on a channel, not just frames addressed to it, which is what makes passive sniffing possible.

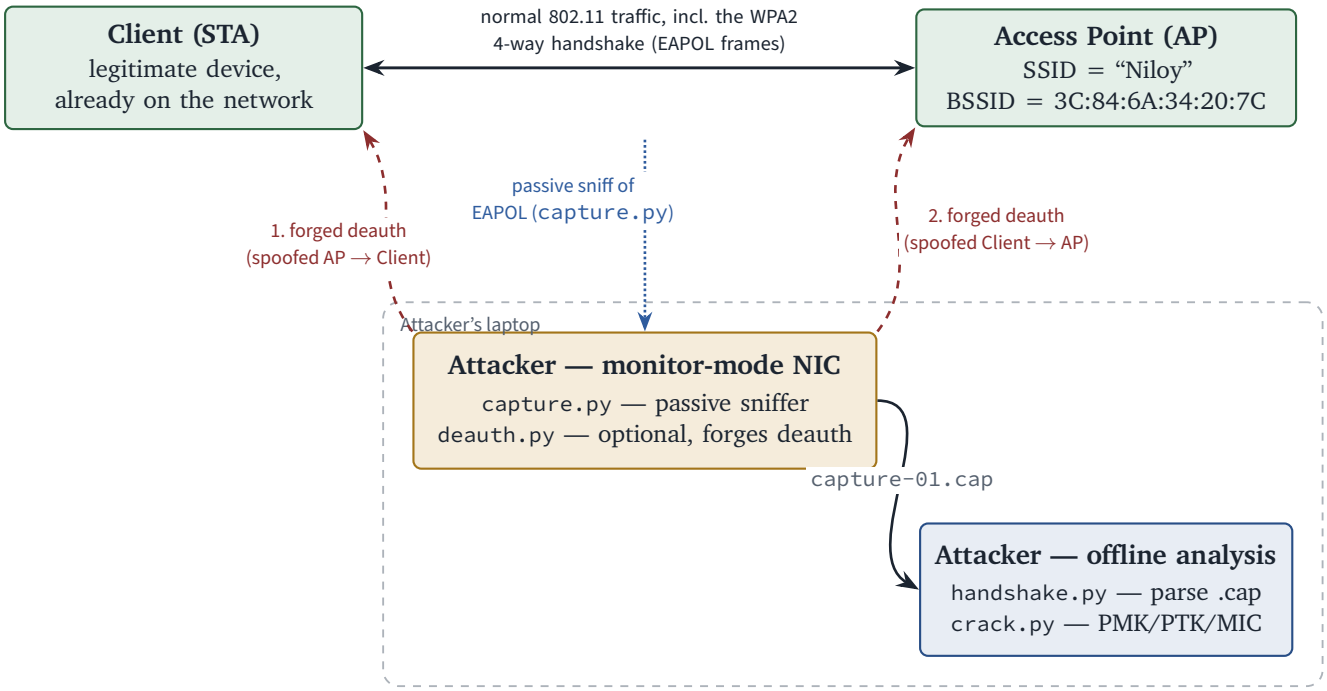


Figure 1. Network topology. The attacker plays two separate roles: an on-air sniffer/injector and a purely offline analysis stage. Steps 1–2 (`deauth.py`) are optional — sniffing and cracking still work if the client reconnects naturally, it just takes longer to wait for.

2.1 Components

The attack is five small scripts plus a settings file. Each has a single, sharply-defined job, and the two offline scripts touch no radio at all.

File	Role	Notes
`config.py`	Single settings file: SSID, BSSID, channel, interfaces, file paths, tunables.	The only file meant to be edited by hand.
`capture.py`	Passively sniffs the monitor interface and saves a usable handshake.	Needs `scapy` and root. Never sends deauth frames itself.
`deauth.py`	Optional: forges deauth frames to force a client to reconnect.	Needs `scapy`, root, and `config.CLIENT_MAC` set.
`handshake.py`	Parses the `.cap` file into candidate 4-way handshakes.	Pure standard library. No radio access needed.
`crack.py`	PMK/PTK/MIC crypto plus the `info` / `single` / `wordlist` CLI.	Pure standard library. No radio access needed.
`test_crack.py`	Automated regression tests for the parser and crypto.	Runs against a real, saved capture file.

Table 1. The five scripts and the settings file, with each one's single responsibility.

2.2 The 4-way handshake and its timing

Once a client is connected, the AP and the client run four EAPOL messages back and forth to derive a fresh session key from the shared password without ever transmitting it. Each message carries a *Key Information* field whose bits identify which of the four messages it is (see section 3.2). Figure 2 shows the original protocol together with the optional forged-deauth step, and marks exactly what each piece of code keeps or uses.

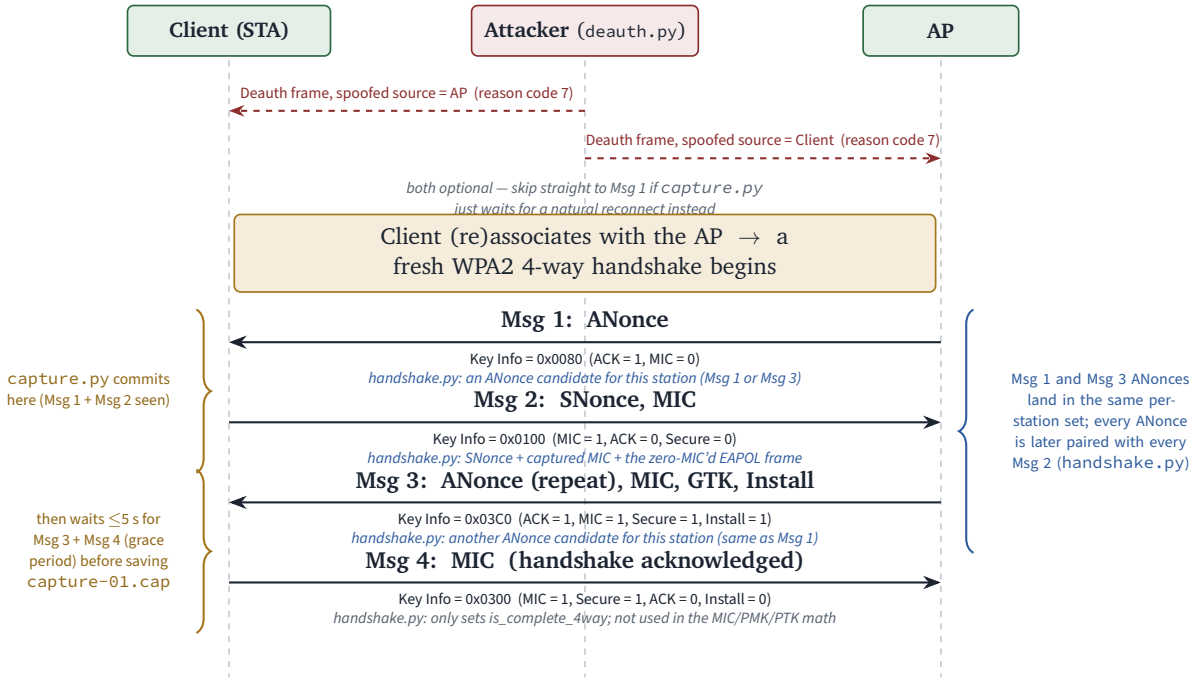


Figure 2. The WPA2 4-way handshake (the *original protocol*, Msg 1–4) together with the attacker's optional `deauth.py` injection (the *attack strategy*, top). Left braces mark exactly when `capture.py` commits and saves; the right brace shows how `handshake.py` pairs the captured nonces.

- **Msg 1** (AP → Client): the AP sends a fresh random number, the **ANonce**.
- **Msg 2** (Client → AP): the client replies with its own random number, the **SNonce**, plus a **Message Integrity Code (MIC)** that only someone who derived the right session key could produce.
- **Msg 3** (AP → Client): the AP confirms and repeats the same ANonce, together with the group key and its own MIC.
- **Msg 4** (Client → AP): the client acknowledges. The handshake is now complete.

`capture.py` commits to a client as soon as it has seen that client's Msg 1 and Msg 2 — that pair alone is enough to attack offline — then waits up to `GRACE_PERIOD_SEC = 5` more seconds for a confirming Msg 3 and Msg 4 before writing the capture file. A capture that only ever got Msg 1+Msg 2 is still fully crackable; it is just not independently confirmed by the AP.

3 Packet and Frame Structure

An EAPOL handshake message does not travel by itself — it is wrapped inside several layers, like an envelope inside an envelope. From the outside in: a normal 802.11 data-frame header, then an LLC/SNAP header that tags the payload as EAPOL, then the EAPOL header, then the EAPOL-Key body that actually carries the ANonce/SNonce and MIC. Figure 3 shows exactly how each layer is sliced, using the same byte offsets and variable names as the code.

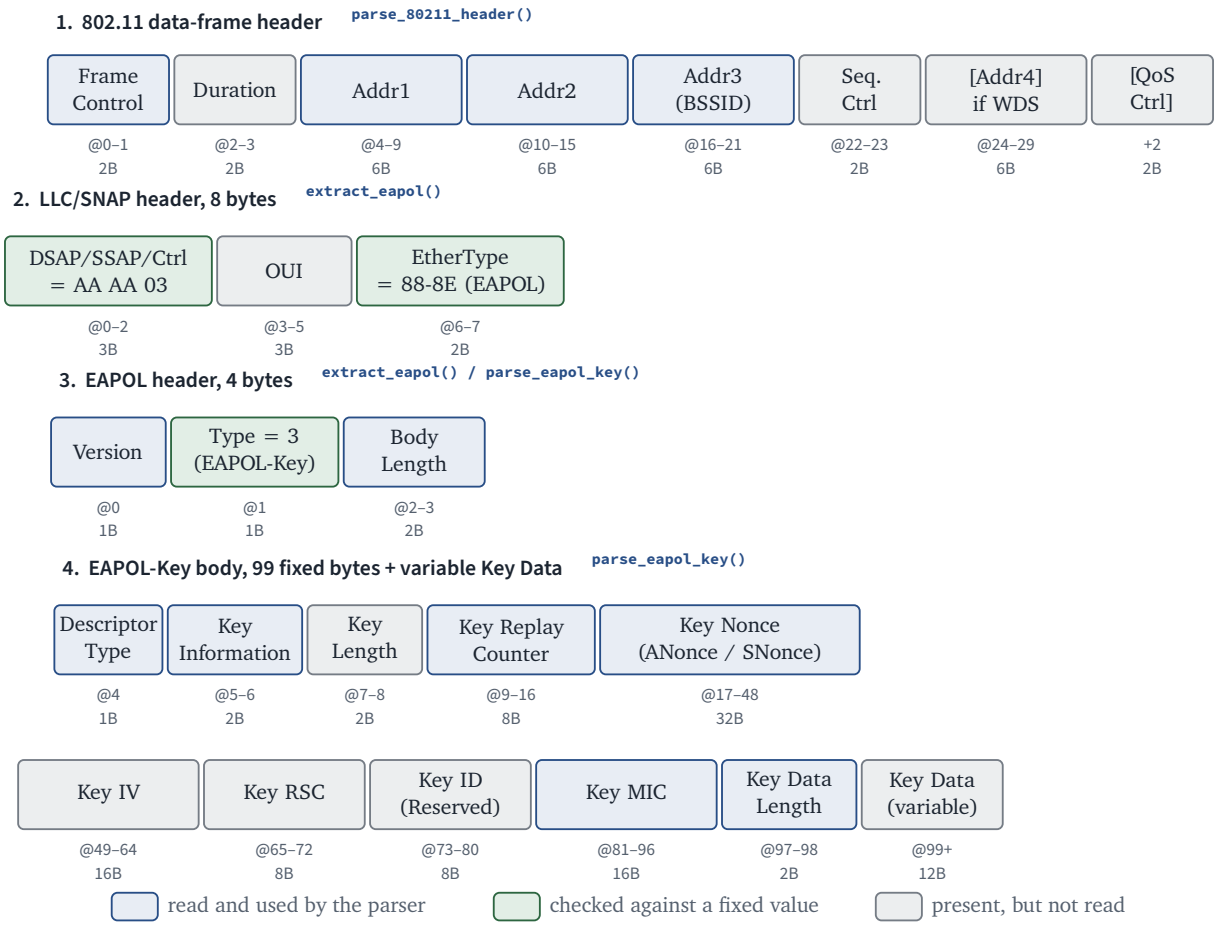


Figure 3. Frame encapsulation from the 802.11 header down to the EAPOL-Key body, with the exact byte offsets and field sizes used by the code. Offsets in layer 4 continue from byte 4 of the EAPOL frame.

3.1 Reading the diagram

- **Layer 1 (802.11 header):** only Frame Control and the three address fields are read. The `to_ds`/`from_ds` bits inside Frame Control tell `parse_80211_header()` which address is the AP and which is the station, and how long the header is (24, 30, 32, or 38 bytes, depending on whether Addr4 and a QoS Control field are present).
- **Layer 2 (LLC/SNAP, 8 bytes):** `extract_eapol()` requires the DSAP/SSAP/Control bytes to read exactly `AA AA 03` and the EtherType to read `88-8E` (the reserved value for EAPOL). Anything else is rejected as "not an EAPOL frame".
- **Layer 3 (EAPOL header, 4 bytes):** the Body Length field tells the parser exactly how many bytes to slice, so a truncated or padded capture frame cannot be mistaken for a complete one.
- **Layer 4 (EAPOL-Key body):** `parse_eapol_key()` rejects anything shorter than 99 bytes outright. The Key Nonce field (32 bytes) is the ANonce on Msg 1/Msg 3 and the SNonce on Msg 2. Three fields in the middle — Key IV, Key RSC, and Key ID/Reserved — exist in every real EAPOL-Key frame but are never read by this parser; they carry no information the MIC check needs.

3.2 The Key Information field

Key Information is a 2-byte bit field. This project reads five things out of it: four single flag bits, and a 3-bit descriptor-version number. Figure 4 shows the exact bit positions, matching the

constants defined at the top of `handshake.py`.

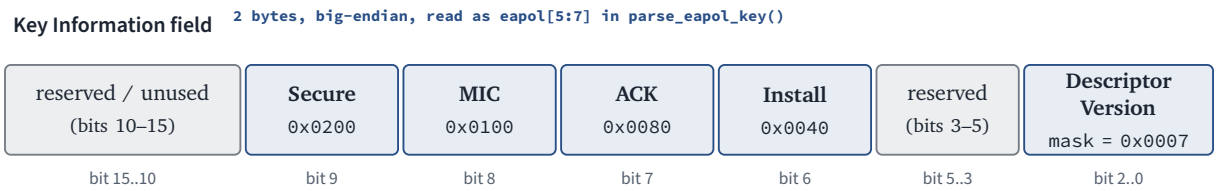


Figure 4. The Key Information bit field, decoded exactly as in `handshake.py`. `classify_message()` names each message from these flags: **Msg 1** = ACK & !MIC; **Msg 2** = MIC & !ACK & !Secure; **Msg 3** = ACK & MIC & Secure & Install; **Msg 4** = MIC & Secure & !ACK & !Install. Any other combination classifies as 0 and is ignored.

`classify_message()` combines these four flags to name each message. This is the exact truth table the code implements (and the exact values exercised by `test_crack.py`'s `test_classify_message`):

Message	ACK	MIC	Secure	Install	Condition in code	Key Info
Msg 1	1	0	–	–	`ack and not mic`	`0x0080`
Msg 2	0	1	0	–	`mic and not ack and not secure`	`0x0100`
Msg 3	1	1	1	1	`ack and mic and secure and install`	`0x03C0`
Msg 4	0	1	1	0	`mic and secure and not ack and not install`	`0x0300`

Table 2. The message-classification truth table implemented by `classify_message()`. A dash means the bit is not tested for that message.

Any other combination of bits classifies as `0` and is ignored — it cannot belong to a standard 4-way handshake, so there is nothing for the parser to do with it.

4 Implementation Plan

The pipeline below is the actual sequence of five scripts, in the order they run and the order data flows between them. Figure 5 shows it as a flowchart; the steps underneath name the exact functions involved.

Step 1 — Configure

Every target-specific and file-specific value — `INTERFACE`, `MONITOR_INTERFACE`, `SSID`, `BSSID`, `CHANNEL`, `CLIENT_MAC`, `CAPTURE_FILE`, `WORDLIST_FILE` — lives in `config.py` and nowhere else, so every other script imports it rather than hard-coding a path or address.

Step 2 — Capture (and, optionally, accelerate)

`capture.py` puts a monitor-mode interface to work: for every sniffed packet that carries both a `Dot11` and an `EAPOL` layer, it re-uses `handshake.classify_message()` (the exact same function the offline parser uses — the live sniffer and the offline parser never disagree about what a Msg 1–4 looks like) to label it. Once one client has produced both Msg 1 and Msg 2, that client is locked in as the target; the script then waits up to 5 more seconds for Msg 3/Msg 4 before writing every collected EAPOL packet to `CAPTURE_FILE` with `scapy`'s `wrpcap()`. It never transmits anything itself.

`deauth.py` is a separate, optional process: it crafts two forged 802.11 deauthentication frames with `scapy` (one addressed to the client claiming to be from the AP, one addressed to the AP claiming to be from the client), and sends 10 bursts of both, 100 ms apart. It refuses to run unless `config.CLIENT_MAC` is set, since a deauth frame needs a specific target. Running it in a second terminal while `capture.py` listens is what actually produces a fast, complete capture; running `capture.py` alone just waits for a natural reconnect.

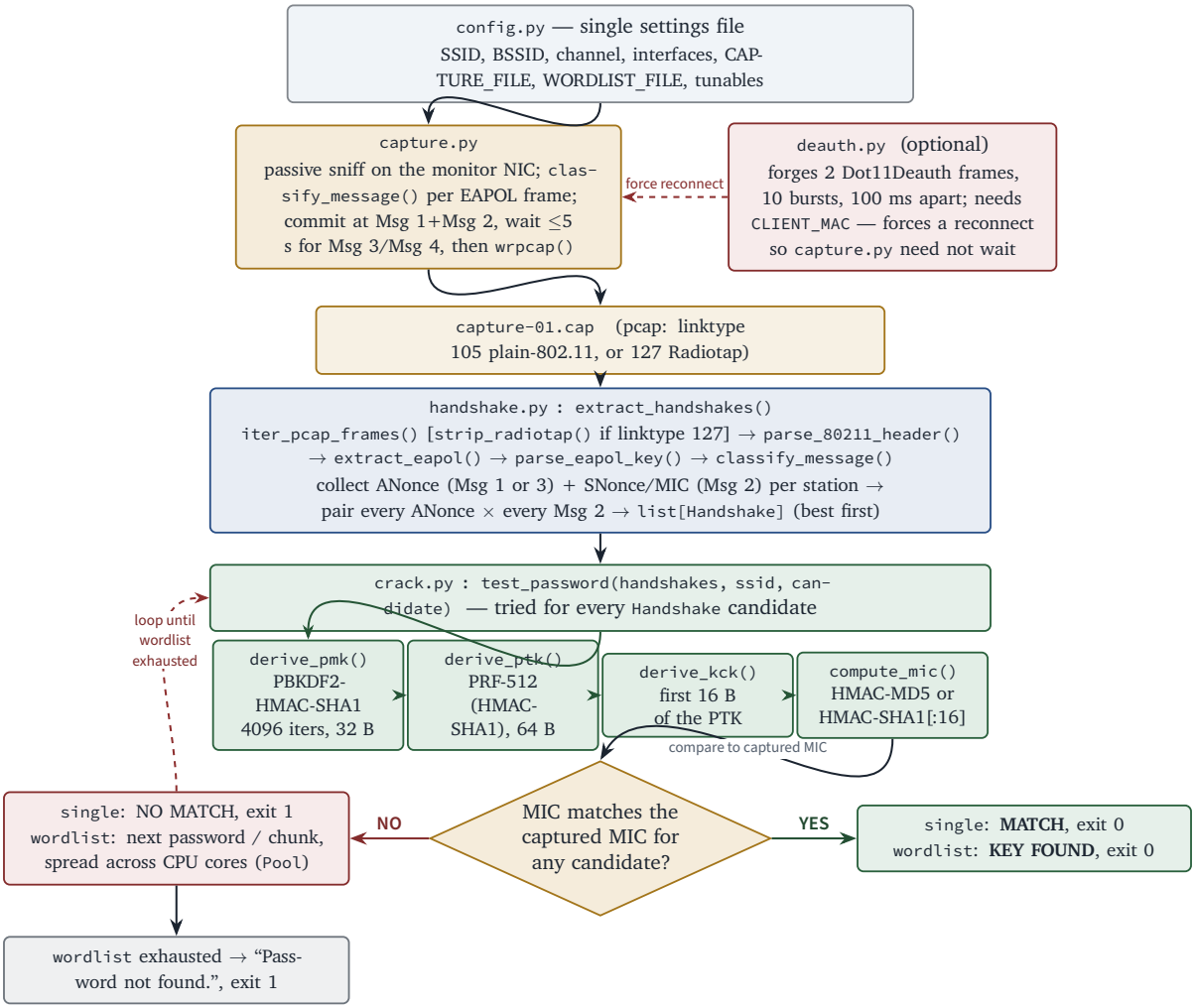


Figure 5. The implementation pipeline, from `config.py` through to a MATCH / NO MATCH decision. The four green boxes are the from-scratch crypto chain inside `test_password()`; the dashed loop returns to `crack.py` for the next candidate until the wordlist is exhausted.

Step 3 — Parse the capture

`handshake.py`'s `extract_handshakes()` turns the saved file into a list of candidate handshakes. It reads the pcap global header to detect linktype 105 (plain 802.11) or 127 (Radiotap-wrapped, what `capture.py`'s live `scapy` sniffing writes) and calls `strip_radiotap()` automatically in the second case. It then filters every frame down to the target BSSID, decodes EAPOL-Key frames, and separates them into two pools per station: every ANonce ever seen (from that station's Msg 1s and Msg 3s — a busy AP can retransmit Msg 1 with a fresh ANonce, and the one Msg 1 a station actually used can be missed by a lossy capture, but Msg 3 always repeats the ANonce it is confirming), and every Msg 2 (SNonce, captured MIC, and the EAPOL frame with its MIC field zeroed out, ready for recomputation). Because there is no way to know in advance which ANonce paired with which Msg 2, every ANonce for a station is paired with every one of that station's Msg 2s, producing one `Handshake` object per pairing. Candidates confirmed by a following Msg 3+Msg 4 are sorted first, since they are independently verified by the AP itself, but every candidate is still tried.

Step 4 — Crack (info / single / wordlist)

`crack.py` tests one candidate password against every `Handshake` object in one call to `test_password()`:

- `derive_pmk(password, ssid)` — PBKDF2-HMAC-SHA1(password, ssid, 4096 iterations, 32 bytes). The SSID is the cryptographic salt, which is why `config.SSID` must match the captured network exactly.
- `derive_ptk(pmk, ap_mac, sta_mac, anonce, snonce)` — PRF-512: a counter-driven loop of HMAC-SHA1 over the label "Pairwise key expansion", both MAC addresses (smaller one first), and both nonces (smaller one first), producing 64 bytes.
- `derive_kck(ptk)` — the first 16 bytes of the PTK: the Key Confirmation Key used only to authenticate handshake messages (a separate slice of the PTK encrypts real data traffic, and is not needed for cracking).
- `compute_mic(kck, frame, descriptor_version)` — HMAC-MD5 (descriptor version 1, WPA/TKIP) or the first 16 bytes of HMAC-SHA1 (version 2, WPA2/CCMP — the case for this project's own capture) over the zero-MIC'd EAPOL frame. Version 3 (AES-128-CMAC, used by 802.11r and WPA3-style networks) is not implemented and raises `ValueError` rather than guessing.

The result is compared byte-for-byte to the MIC actually captured in Msg 2. Three CLI modes drive this:

Mode	Behaviour
`info`	Parses the capture and prints candidate count plus the best candidate's station MAC, replay counter, Complete-4-way flag, descriptor version, and raw ANonce/SNonce/MIC.
`single`	Tests exactly one password. Prints `MATCH` or `NO MATCH`; exit code 0 or 1.
`wordlist`	Streams `WORDLIST_FILE` in chunks of `WORDLIST_CHUNK_SIZE` (2000) passwords, spread across a `multiprocessing.Pool` sized to the CPU count by default, and stops at the first match.

Table 3. The three cracking modes exposed by `crack.py`.

Step 5 — Validate

`test_crack.py` is the regression safety net: parsing tests that run against whatever capture `config.CAPTURE_FILE` currently points to, unit tests for Radiotap stripping and message classification, crypto tests that check output lengths and determinism, and a set of tests pinned to a frozen backup capture (so they keep testing something real even after a fresh `capture.py` run replaces the working file) — covered in detail in section 6.

5 Why the Design Works

The design rests on one fact about the 4-way handshake: it is a *verifiable* proof. Everything the AP and client use to build and check the MIC is either public or is carried, in the clear, inside the handshake itself. So an attacker who guesses the password can reproduce the entire key derivation offline and see, byte-for-byte, whether the guess was right — exactly what the AP would do, but without the AP.

> **The cryptographic chain is deterministic.** Given a password and the captured values, PMK → PTK → KCK → MIC is a fixed sequence of standard primitives (PBKDF2-HMAC-SHA1, PRF-512, HMAC-SHA1/MD5). The right password *must* regenerate the captured MIC; a wrong password will not, except with cryptographically negligible probability. There is no tolerance, tuning, or guesswork in the check itself.

- **The capture is sufficient.** A single Msg 1 + Msg 2 pair already carries every input the check needs: the ANonce, the SNonce, both MAC addresses, the descriptor version, and the captured MIC over a frame we can zero and recompute. The SSID (the PBKDF2 salt) is known from `config.py`. Nothing else is required — which is why the attack is fully offline and needs the AP only once, passively.
- **The parser is robust to real captures.** Linktype auto-detection (105 vs. 127) and automatic Radiotap stripping mean the same code reads both airodump-ng-style and live-`scapy` captures. Pairing *every* ANonce with *every* Msg 2 per station guarantees that packet loss or a Msg 1 retransmission cannot hide the one valid pairing — if a crackable handshake was captured, it becomes one of the candidates tried.
- **The same classifier runs online and offline.** `capture.py` and `handshake.py` call one `classify_message()`, so a handshake the sniffer accepts is exactly one the parser can re-read — there is no drift between what is captured and what is crackable.
- **It is empirically confirmed.** On a real capture, this project's from-scratch crypto and `aircrack-ng` independently recover the *same* password from the *same* bytes (section 6). Two independent implementations agreeing is the strongest evidence that the PMK/PTK/MIC math is correct, not merely self-consistent.

6 Expected Outcome of a Successful Attack

Because every stage of this pipeline prints its own status, a successful run is easy to recognise concretely at each stage — no external tool's output has to be interpreted.

6.1 During capture

`capture.py` prints one line per handshake message it recognises, and once Msg 1+Msg 2 are in hand for one client, commits to it and reports whether the grace period also caught a confirming Msg 3/Msg 4 before writing the file:

```
[+] Msg1/4 from 86:09:70:ae:a2:f3 (have [1]/4 for this client)
[+] Msg2/4 from 86:09:70:ae:a2:f3 (have [1, 2]/4 for this client)
[+] Saved 4 EAPOL frame(s) to capture-01.cap for client 86:09:70:ae:a2:f3 (complete 4-way)
```

6.2 During cracking

- `crack.py info` prints the candidate count and the best candidate's details (station MAC, replay counter, Complete 4-way flag, descriptor version, raw ANonce/SNonce/MIC in hex) — the sanity check that a usable handshake actually exists before spending time on a wordlist.
- `crack.py single "27111998"` prints exactly `[+] MATCH -> password is:  27111998` and exits 0 on a correct guess, or `[-] NO MATCH` and exits 1 otherwise.
- `crack.py wordlist` prints a progress line roughly every 20,000 candidates, then either `[+] KEY FOUND! -> <password>` and exits 0 the moment any worker matches, or, after the whole file is exhausted, `[-] Password not found in wordlist.`

6.3 A concrete, checked example

`test_crack.py`'s `TestAgainstKnownGoodCapture` class is a real, working example of exactly this outcome, pinned to a frozen backup capture so it cannot silently stop testing anything: it asserts that `test_password(handshakes, "Niloy", "27111998")` returns `True` for that capture's real password, and that an obviously wrong password returns `False`. The correct-password result was cross-checked independently against `aircrack-ng` on the same capture and wordlist, which agrees. This double confirmation — this project's own from-scratch crypto, and an established external

tool, reaching the same answer on the same bytes — is the strongest evidence in the repository that the PMK/PTK/MIC implementation is correct, not just internally consistent.

6.4 What a found password actually means

A successful crack recovers the plaintext WPA2-PSK passphrase for the target network. From that point, no further capture or cracking is needed: the same passphrase, together with a captured or future ANonce/SNonce pair, lets an attacker derive the session keys needed to join the network as a normal peer, or to decrypt traffic they can also capture the surrounding handshake for. The password itself is also very often reused by the same person elsewhere, which is a real risk independent of this specific Wi-Fi network.

7 Defense Ideas

The three ideas below are chosen because each blocks a specific mechanism this project actually relies on, rather than being generic advice.

1. Move to WPA3-SAE with Protected Management Frames (PMF / 802.11w)

WPA3's SAE ("Dragonfly") handshake replaces the PSK 4-way handshake with a zero-knowledge-style exchange: capturing it gives an attacker nothing to run an offline dictionary attack against, because it is not designed so that a captured transcript plus a guessed password can be checked offline the way `compute_mic()` does here. Mandatory PMF also cryptographically authenticates management frames, including deauthentication frames — so the forged frames `deauth.py` sends would simply be rejected as unauthenticated, closing off the "force a reconnect on demand" path this project depends on to avoid a slow, passive wait.

2. Use a long, random passphrase

PBKDF2 at 4096 iterations makes each individual guess deliberately slow, but that only matters if the search space is large. The password recovered above, `27111998`, has the exact shape of a birthdate (DDMMYYYY) — precisely the kind of short, guessable, personally-meaningful string that a wordlist attack like `crack.py`'s is built to find quickly across many CPU cores. A long passphrase with real entropy (a random string from a password manager, or several random unrelated words) keeps this same offline attack computationally infeasible — the iteration count buys time per guess, but only passphrase entropy buys enough guesses to matter.

3. Detect and rate-limit deauthentication floods

`deauth.py`'s own signature — ten bursts of two forged deauth frames, 100 ms apart, aimed at one client and the AP — is exactly the kind of pattern a wireless intrusion detection system (WIDS) or a modern AP's built-in management-frame protection can flag and rate-limit. Doing so does not fix the underlying cryptographic weakness, but it removes the accelerant: without `deauth.py`, `capture.py` alone has to wait for a client to reconnect on its own, which can take a while and gives a network operator more time to notice unusual channel activity before a usable handshake is ever captured.